



UNIVERSIDAD DE MONTERREY

Práctica 15

Microprocesadores

M.C. Alejandro Omar Reyes Guía

Nombre: Eugenio Romo García Mat: 612348

Nombre: Ricardo Garza Benítez Mat: 645972

"Damos nuestra palabra de que hemos hecho esta actividad con integridad académica"

Monterrey, N. L., a 23 de junio del 2026

Introducción

En el desarrollo de sistemas embebidos es frecuente encontrar escenarios donde un solo microcontrolador no es suficiente para atender todas las tareas de un sistema: los recursos de memoria, periféricos o pines de E/S pueden agotarse antes de que la aplicación esté completa. Una solución elegante a este problema es distribuir las tareas entre dos o más microcontroladores que se comunican entre sí mediante un protocolo serie. El protocolo UART (Universal Asynchronous Receiver/Transmitter) es uno de los más sencillos y robustos para este propósito, ya que requiere únicamente dos líneas, TX y RX, para establecer un canal bidireccional de comunicación asíncrona.

En esta práctica se implementó una comunicación UART entre dos microcontroladores PIC16F887. El primero, denominado maestro, lee la posición de un potenciómetro mediante su convertidor ADC y convierte el valor de diez bits a un ángulo en grados entre 0 y 180, que transmite por el módulo EUSART hacia el segundo PIC. El segundo, denominado esclavo, recibe el byte de ángulo por su propia UART, lo convierte a un ancho de pulso en microsegundos y genera la señal de control para un servomotor SG90 mediante el Timer1. De esta manera, el giro del potenciómetro en la tarjeta maestra se traduce directamente en el ángulo del servo conectado a la tarjeta esclava, con ambos PICs operando de forma independiente y coordinada a través de un único cable de datos.

La separación de responsabilidades entre ambos nodos es una decisión de diseño que ilustra claramente las ventajas de la arquitectura distribuida: el PIC maestro se dedica exclusivamente a la adquisición analógica y la transmisión serie, mientras que el PIC esclavo gestiona la generación precisa de la señal PWM para el servo sin necesidad de conocer cómo se obtuvo el ángulo. Esta independencia permite, en principio, sustituir cualquiera de los dos nodos o ampliar el sistema sin modificar el otro, siempre que se respete el protocolo de comunicación acordado.

Marco Teórico

Protocolo UART y módulo EUSART del PIC16F887

UART es un protocolo de comunicación serie asíncrono que transmite los datos bit a bit a una velocidad acordada entre emisor y receptor, denominada baudrate. Una trama UART estándar comienza con un bit de inicio en nivel bajo, seguido de ocho bits de datos (LSB primero), un bit de paridad opcional y uno o dos bits de parada en nivel alto. Al no existir señal de reloj compartida, ambos extremos deben estar configurados con exactamente el mismo baudrate para que la sincronización interna de cada receptor sea correcta. El PIC16F887 implementa este protocolo mediante el módulo EUSART, que incluye registros de transmisión (TXREG) y recepción (RCREG) con buffers independientes, lo que permite operación full-duplex. La velocidad de comunicación se configura mediante el registro SPBRG junto con los bits BRGH y BRG16, que seleccionan la fórmula de divisor de baudrate a aplicar [1].

Cálculo del baudrate y registro SPBRG

Con BRGH = 1 (modo de alta velocidad) y BRG16 = 0, la fórmula del baudrate es: $\text{Baudrate} = F_{\text{osc}} / (16 \times (\text{SPBRG} + 1))$. Para una frecuencia de cristal de 16 MHz y un baudrate objetivo de 9600 bps, el valor de SPBRG resulta en 103, con un error de

baudrate inferior al 0.16 %, perfectamente aceptable para comunicación de corto alcance entre dos nodos. Para el PIC esclavo, que en esta práctica opera con un cristal de 8 MHz, el valor de SPBRG necesario para obtener el mismo baudrate de 9600 bps es 51, con un error igualmente despreciable. Es importante que ambos PICs usen exactamente el mismo baudrate para que la decodificación de los bits sea correcta; una discrepancia de más del 3 % acumulada a lo largo de un byte puede causar errores de encuadre detectados por la bandera FERR [1].

Transmisión y recepción EUSART en el PIC16F887

Para transmitir un byte, el firmware escribe el dato en TXREG una vez que la bandera TXIF indica que el registro de transmisión está listo para recibir un nuevo dato. Internamente, el módulo mueve el byte al registro de desplazamiento TSR, que lo envía bit a bit al pin RC6/TX. Del lado receptor, el módulo EUSART acumula los bits recibidos en el pin RC7/RX dentro del registro de desplazamiento RSR y, al completar una trama, transfiere el byte al buffer de recepción RCREG, activando la bandera RCIF. Si llega un segundo byte antes de que el firmware lea el primero, se produce un overrun cuya bandera OERR solo puede limpiarse reiniciando el bit CREN, tal como se implementó en el código del PIC esclavo. La bandera FERR debe leerse antes que RCREG para capturar el error de encuadre asociado al byte que está siendo leído, de lo contrario el valor puede perderse [1].

Mapeo de ángulo a ancho de pulso de servo

El PIC maestro convierte la lectura ADC de diez bits (rango 0–1023) a un ángulo en grados enteros (rango 0–180) mediante la expresión: $\text{angulo} = (\text{adc} \times 180) / 1023$. Este valor cabe en un `uint8_t` y es el único byte transmitido por UART en cada ciclo de 20 ms. El PIC esclavo, al recibirlo, aplica la transformación inversa para obtener el ancho de pulso del servo: $\text{ancho} = 500 + (\text{angulo} \times 2000) / 180$, que mapea 0° a 500 μs y 180° a 2500 μs . El uso de `uint32_t` para los productos intermedios es indispensable en ambas conversiones para evitar desbordamiento de enteros de 16 bits, cuyo límite de 65535 se supera con facilidad al multiplicar valores como $1023 \times 180 = 184\,140$ [1].

Generación de señal de servo con Timer1 a 8 MHz

El PIC esclavo opera con un cristal de 8 MHz, lo que implica un periodo de instrucción de 500 ns ($F_{\text{osc}}/4$). Para mantener la resolución de 2 μs por tick del Timer1 que requiere la generación de la señal de servo, se configura el prescaler en 1:4 en lugar del 1:8 utilizado con 16 MHz: $(500 \text{ ns} \times 4) = 2 \mu\text{s}$ por tick, resultado idéntico. El resto de la lógica de generación de pulso es exactamente la misma que en la práctica anterior: se precarga TMR1H:TMR1L con $65536 - (\text{us}/2)$, se activa el timer y se espera el desbordamiento de TMR1IF. La trama completa de 20 ms se respeta sumando el tiempo en alto más el tiempo en bajo, garantizando así la frecuencia de 50 Hz requerida por el SG90 [1].

Simulación en Proteus

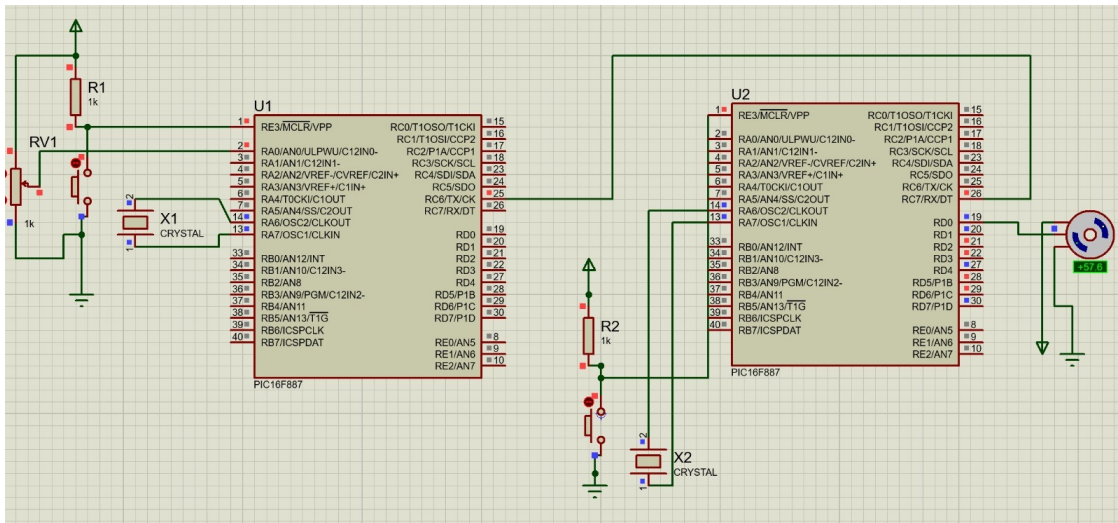
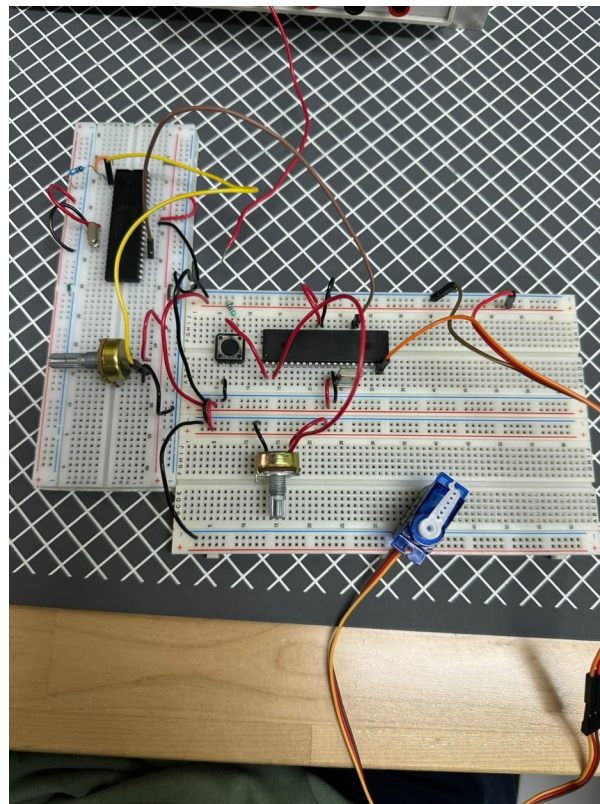


Foto del circuito en Físico



Resultados

La comunicación UART entre ambos PIC16F887 funcionó correctamente desde la primera prueba. El PIC maestro enviaba un byte de ángulo cada 20 ms a través de su pin RC6/TX; ese byte era recibido por el pin RC7/RX del PIC esclavo, el cual lo procesaba de inmediato para actualizar la señal de control del servo. Al rotar el potenciómetro conectado al maestro, el eje del servomotor SG90 seguía la posición de manera proporcional y fluida, confirmando que el mapeo ADC → ángulo → ancho de pulso se ejecutaba sin pérdida de información en ninguna de las dos etapas.

Se verificó que los extremos del rango eran alcanzables: con el potenciómetro en posición mínima el servo se posicionaba en 0° y con la posición máxima llegaba a 180°. No se observaron errores de encuadre ni overruns durante las pruebas normales de operación, lo que indica que el baudrate de 9600 bps fue calculado correctamente en ambos PICs a pesar de operar con cristales de diferente frecuencia: 16 MHz en el maestro y 8 MHz en el esclavo. La diferencia en frecuencia de cristal no afectó la comunicación gracias a que el SPBRG se ajustó de forma independiente en cada firmware para producir exactamente el mismo baudrate.

Un aspecto que llamó especialmente la atención fue la sencillez física del enlace de comunicación: la conexión entre ambos PIC se reduce a un único cable entre RC6 del maestro y RC7 del esclavo, más la referencia común de tierra. Con ese solo hilo de datos fue posible transmitir información de control en tiempo real y coordinar el comportamiento de dos microcontroladores que operan de forma completamente independiente. Esto ilustra de manera muy directa cómo la comunicación serie permite expandir la capacidad funcional de un sistema distribuido con un impacto mínimo en el cableado físico.

Desde el punto de vista del diseño de software, la separación de roles entre el nodo maestro y el nodo esclavo resultó en un código limpio y fácil de mantener en cada PIC. El maestro no necesita saber nada sobre servomotores, ni el esclavo sobre potenciómetros o ADC; cada uno opera dentro de su dominio con una interfaz mínima de un byte por trama. Esta arquitectura podría escalar fácilmente a redes de más nodos utilizando el mismo principio de comunicación serie.

Conclusiones Individuales

Ricardo Garza Benítez

Esta práctica fue de las que más me hicieron pensar en sistemas reales. Ver que dos PICs se comunican con un solo cable y que con eso prácticamente se duplica la capacidad del sistema, tanto en memoria como en periféricos disponibles, cambia bastante la perspectiva sobre cómo diseñar un proyecto embebido. Antes de esta práctica mi primer instinto al quedarme sin pines o memoria sería buscar un microcontrolador más grande; ahora considero primero si puedo distribuir la tarea en dos nodos conectados por UART.

Lo que más cuidé al armar el circuito físico fue que ambas tierras estuvieran unidas: al principio olvidé conectar los GND de las dos protoboards y la comunicación no

funcionaba aunque el cableado TX-RX parecía correcto. Una vez hecha esa conexión el sistema respondió de inmediato. Ese detalle me dejó claro que en cualquier comunicación serie entre dispositivos que no comparten alimentación, la referencia común es tan importante como el propio cable de datos.

Eugenio Romo García

Esta práctica me pareció muy ilustrativa porque mostró cómo un protocolo simple como UART puede ser suficiente para coordinar dos microcontroladores en una aplicación real. Lo que más me llamó la atención fue que con transmitir un solo byte por trama, el ángulo del servo, se logra una comunicación funcional y eficiente: no se necesita ningún protocolo complejo ni encabezado de trama cuando el dato tiene un rango bien definido y ambos nodos conocen su significado.

También fue interesante ver que los dos PICs podían tener cristales de diferente frecuencia, 16 MHz y 8 MHz, y aun así comunicarse sin problemas ajustando únicamente el valor de SPBRG en cada firmware. Eso refuerza la idea de que UART es un protocolo muy flexible en cuanto a hardware, ya que cada nodo solo necesita ser capaz de producir el baudrate correcto independientemente de su velocidad de reloj interna.

Referencias

- [1] Microchip Technology. (2009). PIC16F887 Data Sheet. <https://ww1.microchip.com/downloads/en/DeviceDoc/41291D.pdf>
- [2] Tower Pro. (2010). SG90 Micro Servo Datasheet. <http://www.towerpro.com.tw/product/sg90-7/>
- [3] Merino Pérez, L. A. (2012). Microcontroladores PIC: Diseño práctico de aplicaciones. McGraw-Hill.